

15 图论和网络

15.1 无向图

边没有方向、连接彼此对称——用对称的邻接矩阵描述无向图，矩阵的幂给出途经节点的路径数量



鸢尾花书 · 《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1 无向图中的边没有方向，连接是对称的

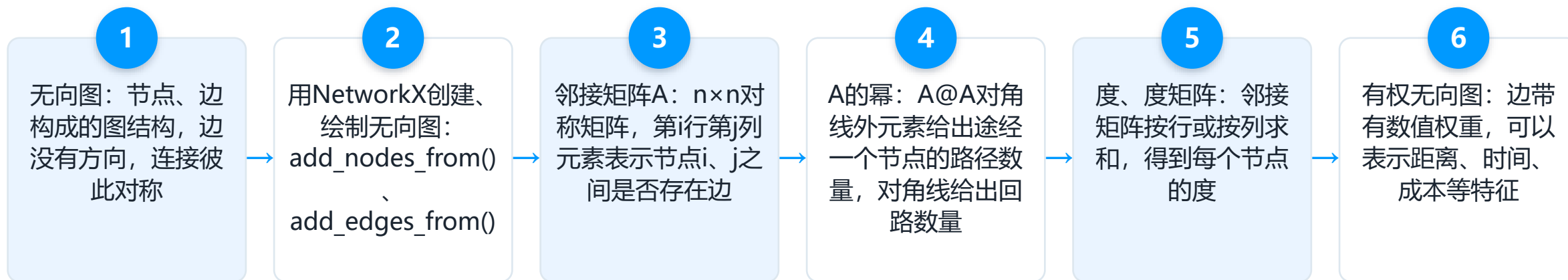
2 无向图的邻接矩阵：对称矩阵

3 邻接矩阵的幂表示节点之间经过多个中介节点的路径数量

4 邻接矩阵行列求和计算节点度

5 有权无向图：边含有权重

从无向图，到邻接矩阵，再到路径数量、度、权重



矩阵是描述和分析图、网络的绝佳工具

① 图论研究由节点和边组成的图结构；机器学习、人工智能、日常生活中，图、网络无处不在

② 本章相当于图论和网络的入门教程，前文学过的线性代数工具在本章随处可见

③ 15.1 无向图：边没有方向，邻接矩阵是对称矩阵（本节）

④ 15.2 有向图：边有方向，邻接矩阵不再对称，建议与本节平行阅读

⑤ 15.3 转移矩阵：描述图上随机游走、状态转移的概率矩阵

⑥ 15.4 拉普拉斯矩阵和谱聚类：结合特征值分解，把图结构用于聚类分析

图 1 • 公园景点之间构成的无向图

- 公园4个景点看作4个节点a、b、c、d；景点间的徒步小径看作边，共5条边：ab、ac、ad、bc、cd，边没有方向

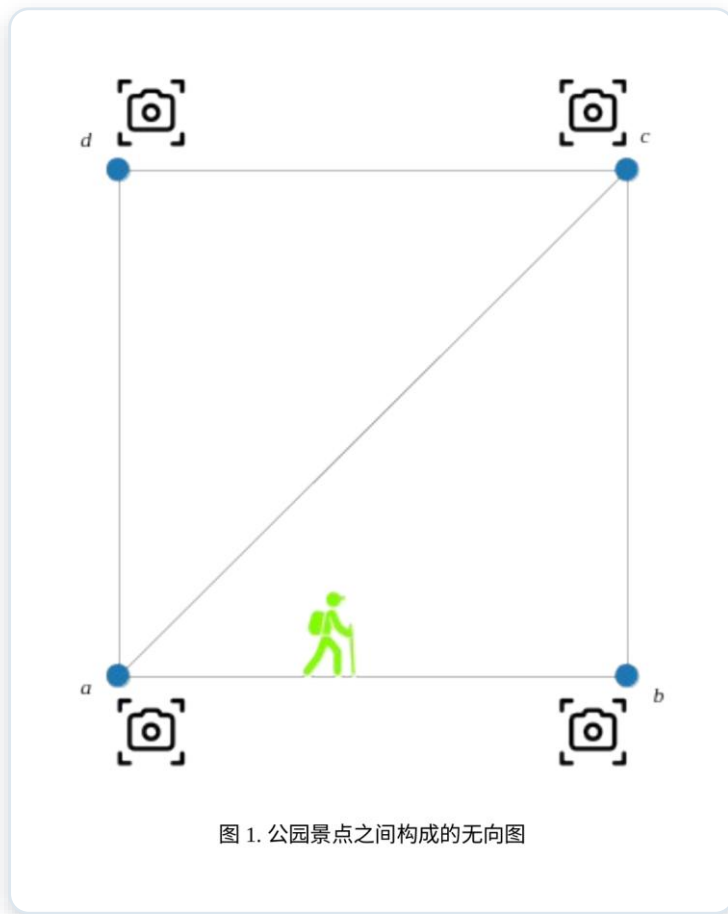


图 1. 公园景点之间构成的无向图

用NetworkX创建、绘制无向图

代码 1. 用NetworkX绘制无向图

- 导入图结构处理库NetworkX (简写nx)
; nx.Graph()创建一个空的无向图对象
- add_nodes_from(['a','b','c','d'])、
add_edges_from([...])分别批量添加节点、边
- 无向图中('a','b')和('b','a')表示同一条边
，没有先后之分
- 字典pos手动指定每个节点的二维坐标;
nx.draw_networkx()一步完成节点、边、
标签的绘制

```
## 初始化
import matplotlib.pyplot as plt
import networkx as nx

## 无向图
undirected_G = nx.Graph()

## 添加节点
undirected_G.add_nodes_from(['a', 'b', 'c', 'd'])

## 添加边
undirected_G.add_edges_from([('a', 'b'),
                              ('a', 'c'),
                              ('a', 'd'),
                              ('b', 'c'),
                              ('c', 'd')])

## 定义节点坐标
pos = {'a': [0, 0],
       'b': [1, 0],
       'c': [1, 1],
       'd': [0, 1]}

## 可视化
plt.figure(figsize = (6,6))
nx.draw_networkx(undirected_G, pos = pos, node_size = 188)
```

无向图可以用邻接矩阵（对称矩阵）表示

$$A = [0 \ 1 \ 1 \ 1; 1 \ 0 \ 1 \ 0; 1 \ 1 \ 0 \ 1; 1 \ 0 \ 1 \ 0]$$

- 对于n个节点的无向图，邻接矩阵是 $n \times n$ 对称矩阵A；第i行第j列元素表示节点i、j之间是否存在边（1为存在，0为不存在）
- 无向图边没有方向，因此A关于主对角线对称：第i行第j列元素等于第j行第i列元素
- A的主对角线元素均为0，因为图1每个节点都不存在到自身的边；这种边如果存在，叫做自环（self-loop）

图 2 • 无向图邻接矩阵的每个元素

- 逐一验证矩阵元素与边的对应关系：a、b之间有边，A第1行第2列、第2行第1列均为1；c、d之间无边，对应元素为0

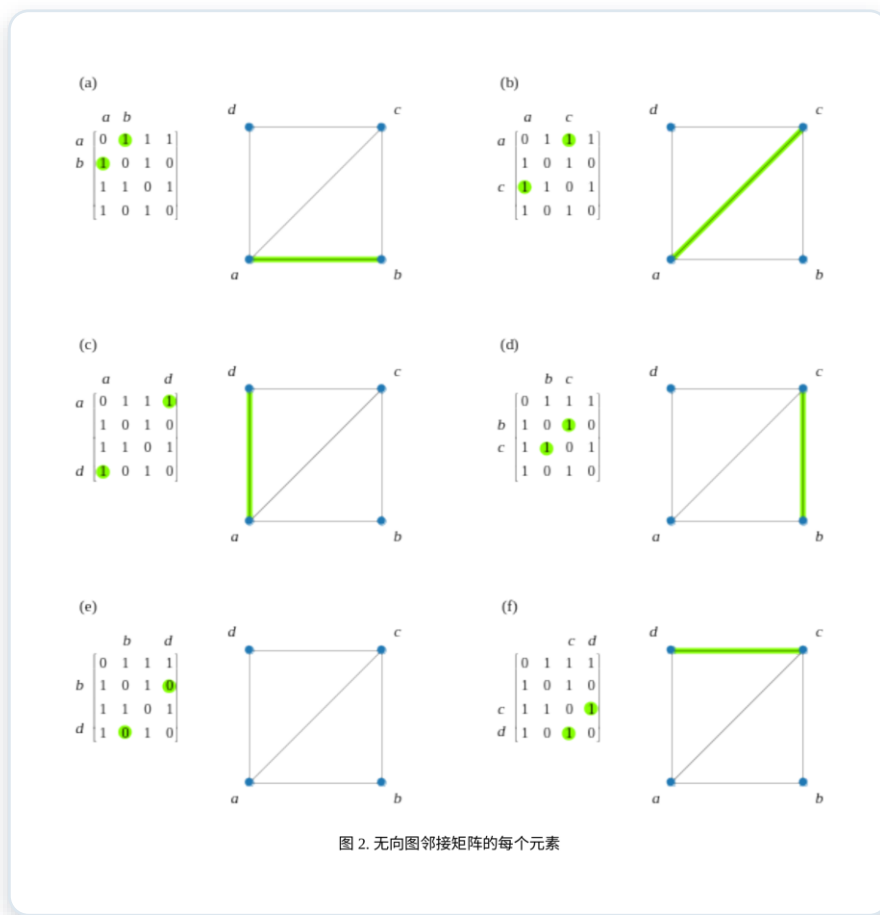


图 2. 无向图邻接矩阵的每个元素

表 1 · 4个节点构造的几种无向图及邻接矩阵（上）

- 同样4个节点a、b、c、d可以构成不同结构的无向图，每种结构对应一个不同的对称邻接矩阵，请大家逐一分析每个图与矩阵的对应关系

表 1. 4个节点构造的几种无向图及邻接矩阵，表格示例来自《数据有道》

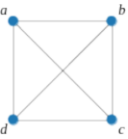
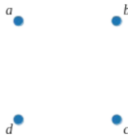
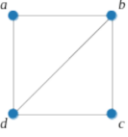
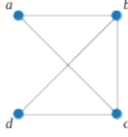
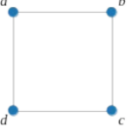
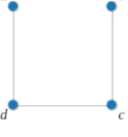
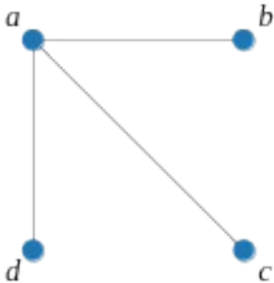
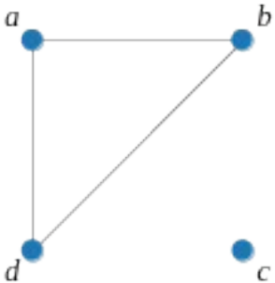
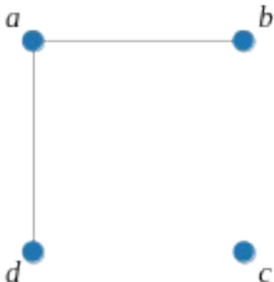
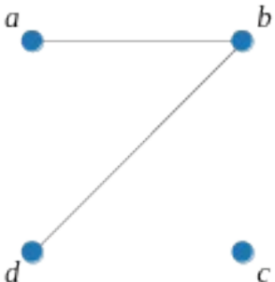
无向图	邻接矩阵	无向图	邻接矩阵
	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$

表 1 · 4个节点构造的几种无向图及邻接矩阵（续）

- 继续分析表格后两行的无向图与对应邻接矩阵，注意对比不同连接方式下矩阵中1的分布规律

	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$

从邻接矩阵构造无向图

代码 2. 从邻接矩阵到无向图

- 用numpy.array()直接定义一个对称的4×4邻接矩阵adjacency_matrix
- nx.Graph(adjacency_matrix, nodetype=int)自动根据矩阵中值为1的位置建立图的边
- 字典mapping = 0:'a', 1:'b', 2:'c', 3:'d'把默认的数字节点编号重命名为字母
- nx.relabel_nodes()完成节点重命名, 让图更易读、便于标注

```
## 初始化
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np

## 定义邻接矩阵
a adjacency_matrix = np.array([[0, 1, 1, 1],
                                [1, 0, 1, 0],
                                [1, 1, 0, 1],
                                [1, 0, 1, 0]])

## 创建无向图
# 用邻接矩阵创建无向图
b undirected_G = nx.Graph(adjacency_matrix, nodetype=int)

# 构造节点映射
c mapping = {0: 'a', 1: 'b', 2: 'c', 3: 'd'}

# 重命名节点
d undirected_G = nx.relabel_nodes(undirected_G, mapping)

## 定义节点坐标
pos = {'a': [0, 0],
        'b': [1, 0],
        'c': [1, 1],
        'd': [0, 1]}

## 可视化
plt.figure(figsize = (6,6))
nx.draw_networkx(undirected_G, pos = pos,
                  node_size = 188, with_labels=True)
```

图 3 • a直达三个节点，邻接矩阵A的第1行

- 邻接矩阵A第1行（除主对角线外）值为1的位置，对应节点a可以不途径任何节点直接到达的节点：b、c、d

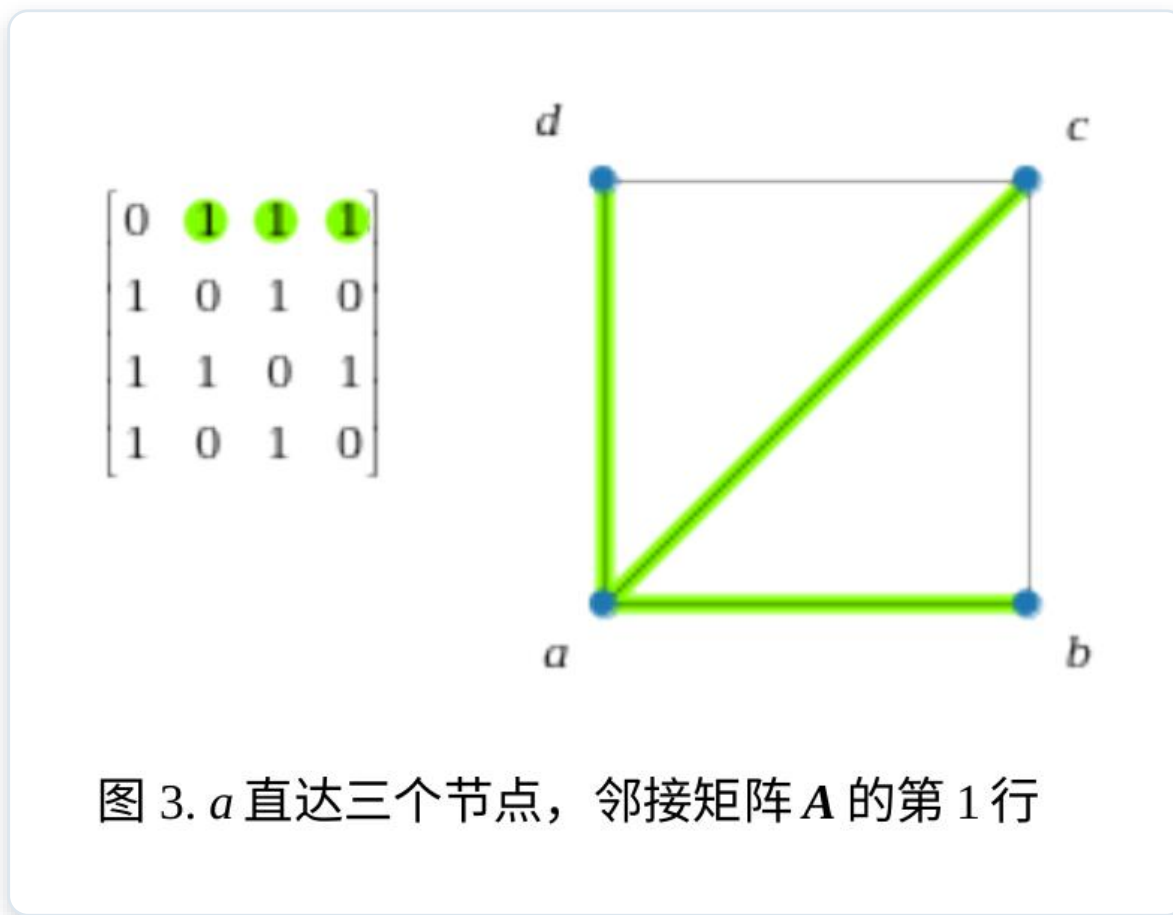


图 4 • a直达三个节点，邻接矩阵A的第1列

- 邻接矩阵A的第1列给出相同信息；由于A对称，行、列可以互换解读，结果一致

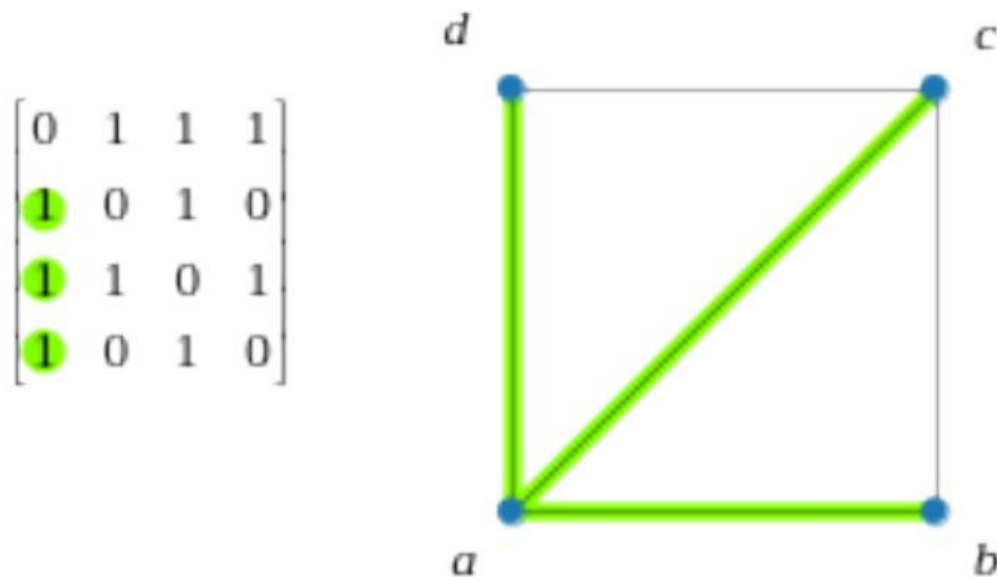


图 4. a 直达三个节点，邻接矩阵 A 的第 1 列

图 5 · 节点a途经一个节点到达c的路径

- $A @ A$ 结果第1行第3列（或第3行第1列）元素值为2，对应节点a途经一个中间节点到达c的两条路径： $a \rightarrow b \rightarrow c$ 、 $a \rightarrow d \rightarrow c$

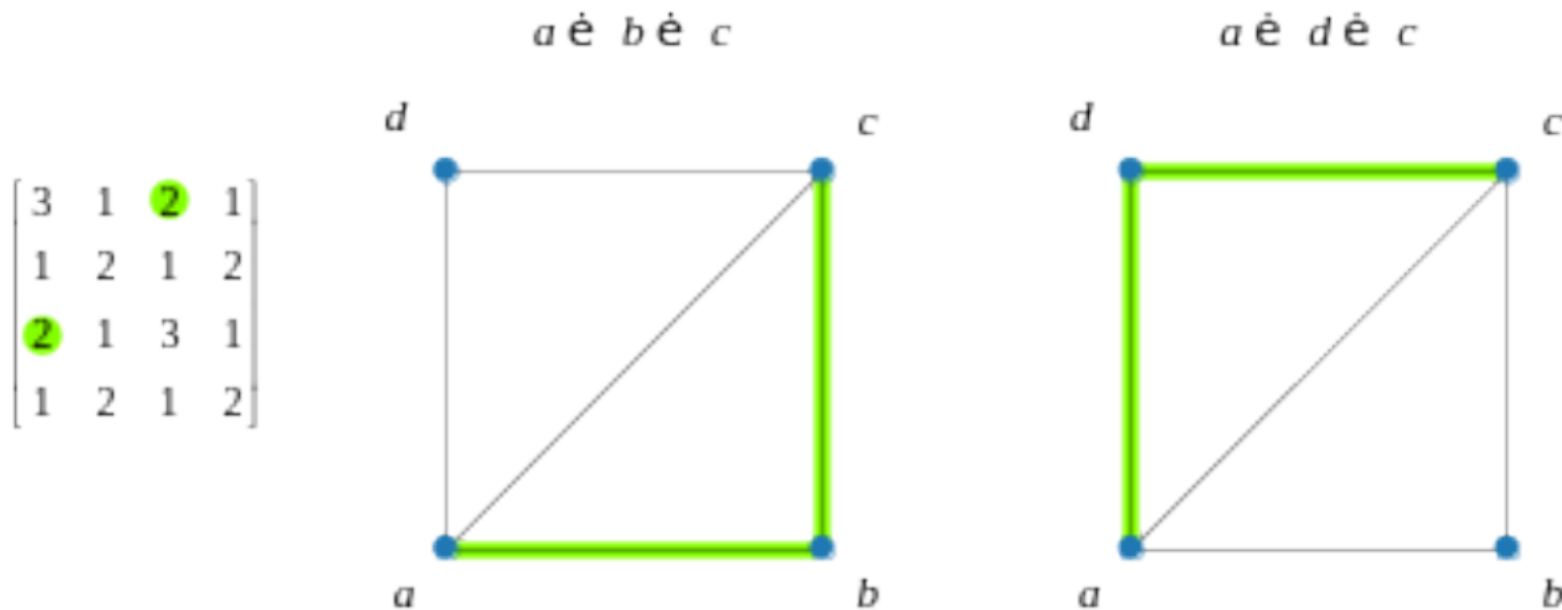


图 5. 节点 a 途经一个节点到达 c 的路径

图 6 · 节点a途经一个节点再回到a的路径

- A^2 主对角线元素代表从某节点出发、途经一个节点又回到自身的路径数量：节点a途经一个节点再回到a的路径有3条

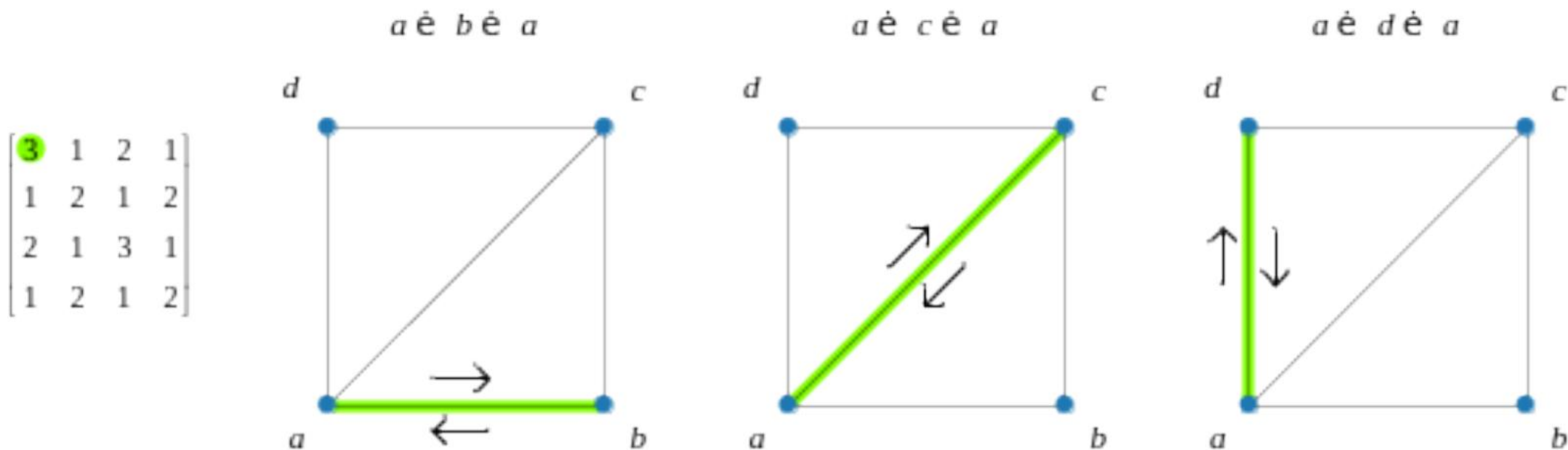


图 6. 节点 a 途经一个节点再回到 a 的路径

邻接矩阵按行或按列求和，就是每个节点的度

① 度 (degree)：一个节点与图中其他节点相连的边数，反映节点的活跃程度

② 度矩阵D：记录所有节点度数的对角方阵，主对角线之外元素全为0

③ 按列求和： $\mathbf{1}^T \mathbf{A}$ 得到每个节点的度，再用diag()构造度矩阵D

④ 按行求和：由于无向图邻接矩阵A对称，按行求和 $\mathbf{A} \mathbf{1}$ 的结果与按列求和相同

图 7 • 无向图的度矩阵

- 图1的度矩阵 $D=\text{diag}(3,2,3,2)$ ，对应节点a、b、c、d的度分别是3、2、3、2；社交网络中度数高像“交际花”，度数低则偏好独处

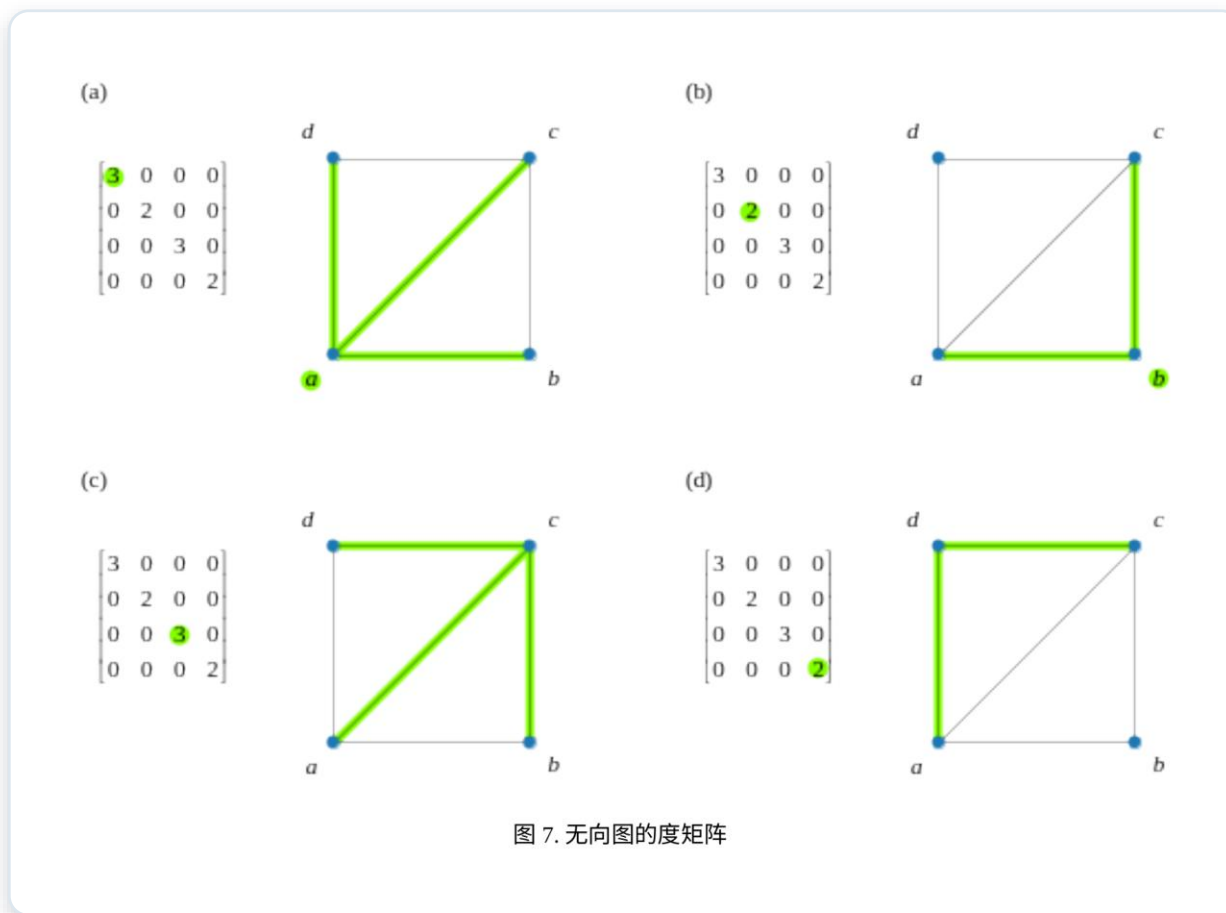
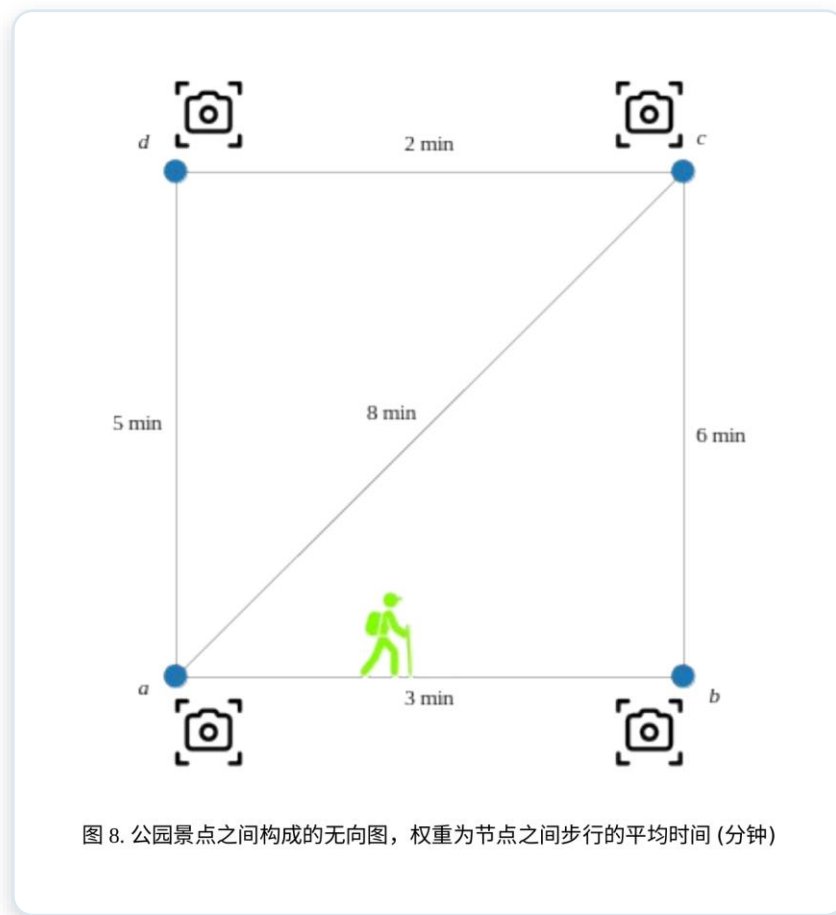


图 8 • 公园景点之间构成的无向图（权重为步行时间）

- 有权无向图的每条边带有数值权重，比如图中标注的节点间平均步行时间（分钟）；权重也可以表示距离、成本等其它特征



无向图对称，邻接矩阵对称，度矩阵藏在求和里

- ① 无向图的边没有方向，连接关系是对称的
- ② 邻接矩阵 A 是对称矩阵，无自环时主对角线为0，元素表示节点之间是否相连
- ③ 邻接矩阵 A 的 k 次幂，其元素表示节点之间途经 $k-1$ 个中介节点的路径数量
- ④ 度是与某节点相连的边数，度矩阵 D 是记录所有节点度数的对角方阵
- ⑤ 邻接矩阵按列（或按行）求和，就是每个节点的度
- ⑥ 有权无向图的边带有数值权重，可以表示距离、时间、成本等特征

本节关键词

无向图 边没有方向的图，连接关系对称

自环 一条连接某个节点自身的边

度矩阵 记录所有节点度数的对角方阵

邻接矩阵 $n \times n$ 对称矩阵，元素表示节点间是否存在边

度 一个节点与图中其他节点相连的边数

有权无向图 每条边都带有数值权重的无向图

延伸阅读

延伸阅读

复习 · 第3章 分块矩阵乘法

理解邻接矩阵的幂 A^k 背后的计算逻辑

复习 · 第2章 矩阵转置

对称矩阵 $A^T = A$ 是理解无向图邻接矩阵的基础

预告 · 15.2 有向图

边有方向时，邻接矩阵不再对称，建议与本节平行阅读

开源代码仓库

全书代码、图片: <https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 请用NetworkX逐一构建并绘制表1中所有无向图。

Q3 请计算表1中所有无向图的邻接矩阵。

Q5 请自学完全图、空图这两个概念，并指出表1中的完全图、空图分别是哪个。

Q7 用Python验证：图1中节点a、b、c、d的度之和，恰好等于边数的2倍。

Q9 用numpy.trace()计算图1邻接矩阵A@A的迹，并解释这个数值的图论含义。

Q2 请计算表1中所有无向图节点的度。

Q4 请计算表1中所有无向图的度矩阵。

Q6 用Python计算图1邻接矩阵A的A@A@A，并解释其主对角线元素、非对角线元素分别代表什么路径。

Q8 自己设计一个5个节点的无向图，用NetworkX绘制，并计算其邻接矩阵、度矩阵。

习题 Q10 – Q17

Q10 用Python验证：无向图邻接矩阵A按列求和与按行求和的结果完全相同。

Q12 用Python为Q8设计的无向图计算 $A+A@A$ ，并分析其对角线、非对角线元素的含义。

Q14 用NetworkX的`nx.degree()`函数直接计算图1中每个节点的度，并与手算结果对比。

Q16 解释有权无向图中，权重除了表示步行时间，还可以表示哪些特征。

Q11 解释为什么邻接矩阵的主对角线元素均为0，除非图中存在自环。

Q13 举例说明生活中还有哪些场景可以用无向图建模，并画出对应的无向图。

Q15 用Python构造图1的有权版本（图8），并用`nx.draw_networkx_edge_labels()`标注权重。

Q17 用Python编写函数，判断一个给定的方阵是否是合法的无向图邻接矩阵（对称、主对角线为0）。

习题 Q18 – Q25

Q18 对于Q8设计的无向图，计算从任意两个节点之间途经不超过2个节点的路径数量。

Q20 用Python比较完全图（表1中所有节点两两相连）与空图（无边）的度矩阵有何特点。

Q22 解释节点度数在社交网络分析中的实际意义，并举例说明度数最高节点的作用。

Q24 结合本节内容，说明为什么矩阵、矩阵分解是分析图、网络结构的重要工具。

Q19 解释为什么邻接矩阵A的k次幂中，元素值会随着k增大而普遍变大。

Q21 用Matplotlib绘制Q8设计的无向图邻接矩阵的热图，并与图的连接结构对照分析。

Q23 用Python验证：一个n个节点的完全图，每个节点的度都等于n-1。

Q25 预习15.2节内容，猜想有向图的邻接矩阵会有什么不同的性质，并说明理由。

10个和AI互动的Prompt

1 解释无向图和有向图的本质区别，以及各自对应的邻接矩阵有什么不同。

2 解释为什么无向图的邻接矩阵一定是对称矩阵。

3 解释邻接矩阵的幂 A^2 、 A^3 分别代表什么图论含义。

4 解释度、度矩阵的定义，以及它们在图算法中的作用。

5 举例说明生活中哪些真实网络可以用无向图建模。

10个和AI互动的Prompt

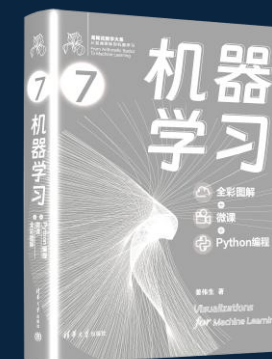
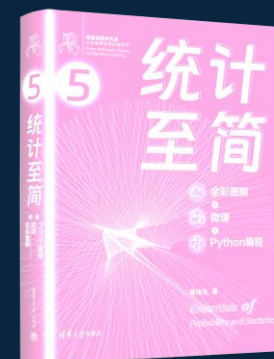
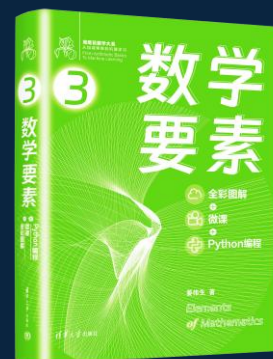
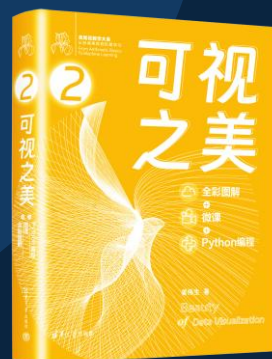
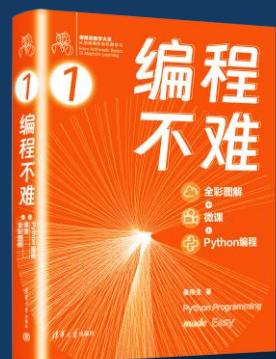
- 6 解释自环的含义，以及自环存在时邻接矩阵会如何变化。
- 7 解释有权无向图和无权无向图的区别，以及权重可以表示哪些特征。
- 8 解释为什么邻接矩阵按行、按列求和都能得到节点的度。
- 9 解释完全图、空图这两个概念，并说明它们的邻接矩阵各有什么特点。
- 10 解释度矩阵和邻接矩阵在计算拉普拉斯矩阵时分别扮演什么角色。

数学

数学基础

线性代数

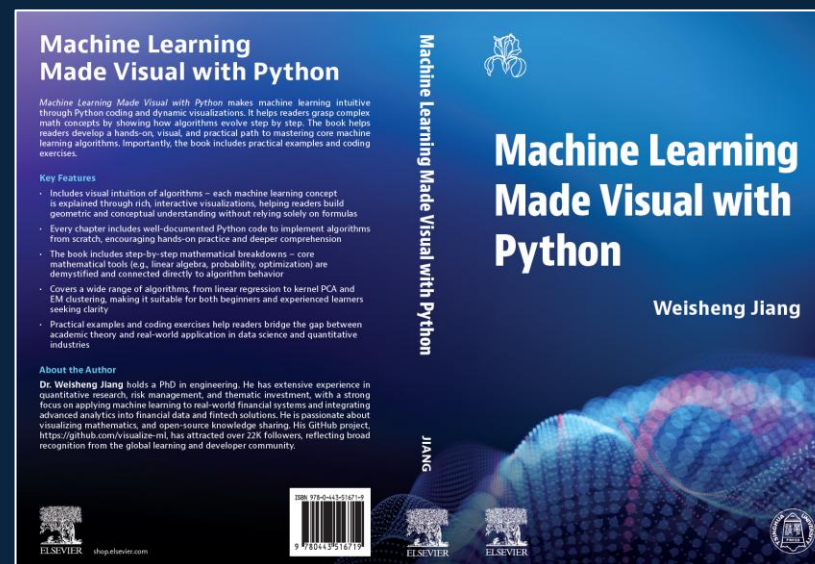
概率统计



Python编程

数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践